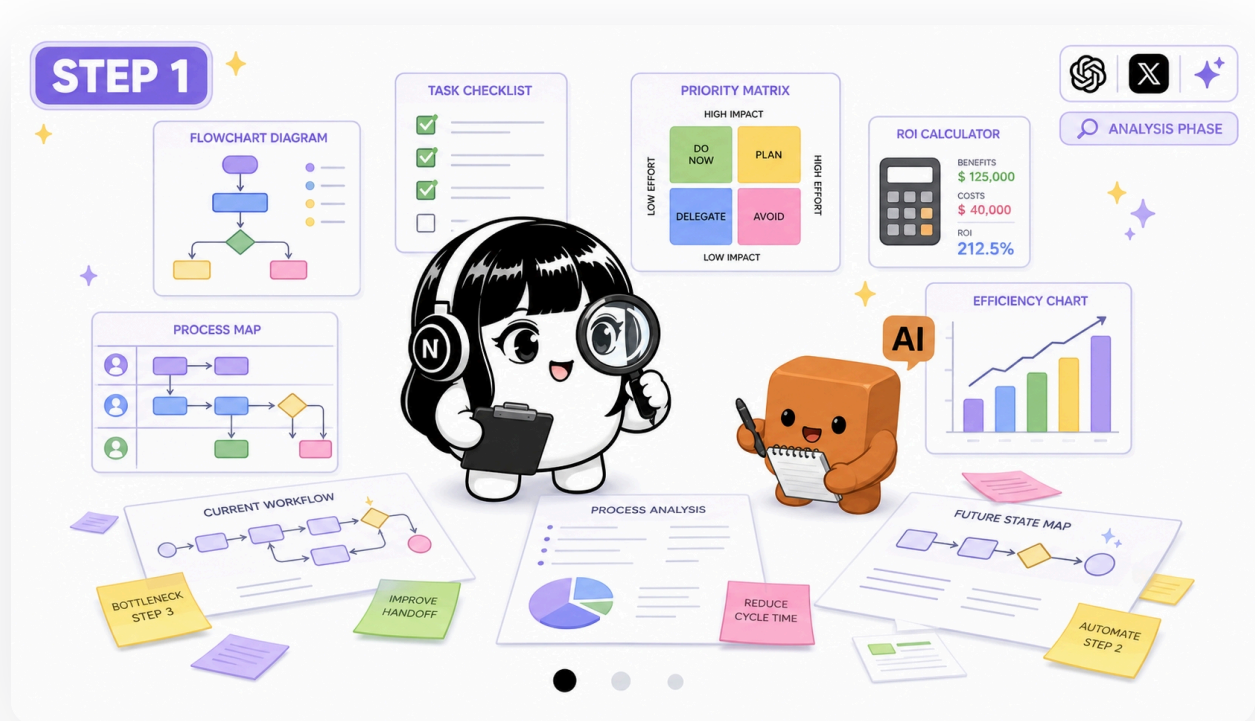


【保存版】初心者向け Claude Codeの教科書——インストールから実務まで全部入り

2026.05.07 15 min read



こんな悩みありませんか？

Claude Codeって聞いたことあるけど、ChatGPTと何が違うの？

ターミナルって時点でもう無理な気がする

インストールしたはいいいけど、何をどう使えばいいかわからない

とりあえず触ってみたけど、トークンだけ消費して何も進まなかった

周りが「Claude Codeヤバイ」って言うてるけど、自分だけ取り残されてる気がする

もしひとつでも当てはまるなら、この記事はあなたのために書きました。

Claude Codeは2025年2月にAnthropicが公開したCLI（コマンドラインインターフェース）ベースのAIコーディングエージェントです。公開から1年以上が経ち、いまやブロの開発者から学生まで、世界中のユーザーが日常の開発ツールとして使っています。

でも「最初の壁」でつまずいて止まってしまう人がものすごく多い。

この記事では、Claude Codeをまったく触ったことがない人でも、インストールから実務レベルの活用まで一気に理解できるように、全部まとめました。

そもそもClaude Codeとは何か

まずここを正確に理解しないと、後の全部がぼやけます。

Claude Codeは、Anthropicが開発した**ターミナル上で動くAIエージェント**です。ChatGPTやClaudeのWebアプリとは根本的に違います。

何が違うかというと、Claude Codeは「**あなたのPC上のファイルを直接読んで、直接編集できる**」こと。

ChatGPTに「このコードを修正して」と頼むと、修正案がチャット画面に表示されます。それをコピーして、自分のエディタに貼り付けて、保存する。この手作業が毎回発生します。

Claude Codeの場合、あなたのプロジェクトフォルダの中に入って、ファイルを読み、理解し、必要な箇所を直接書き換えます。ターミナルコマンドも実行できる。gitも操作できる。テストも走らせられる。

つまりClaude Codeは「チャットで答えを教えてくれるAI」ではなく、「あなたのPCの中で一緒にコードを書いてくれる開発パートナー」です。

ここが分かっていないと「ChatGPTでよくない？」という疑問から抜け出せません。

ChatGPTやCursorとの違い

よく比較されるのがこの3つです。整理します。

ChatGPT / Claude Web版

→ ブラウザ上でAIと会話する。コードの相談はできるが、実行もファイル操作もAI側からはできない。「アドバイザー」。

Cursor / Windsurf (AIEDiタ)

→ VS CodeベースのエディタにAIが内蔵されている。ファイルの読み書きはできるが、基本的にエディタ内の操作に限られる。「AIアシスタント付きエディタ」。

Claude Code

→ ターミナルから起動して、ファイルシステム全体にアクセスできる。コード編集だけでなく、git操作、コマンド実行、環境構築、デプロイまで自律的にやれる。「AIエンジニア」。

CursorやWindsurfが「エディタの中にAIがいる」なら、Claude Codeは「AIそのものが開発者として動く」感覚に近い。

もちろん目的によって使い分けるのがベストですが、Claude Codeの守備範囲の広さは圧倒的です。1つのツールでコードの生成、修正、テスト、git管理、デプロイまで完結する。これがClaude Codeが「ゲームチェンジャー」と呼ばれている理由です。

インストール方法

ここでつまづく人が多いけど、実はめちゃくちゃ簡単です。

必要なもの:

Node.js (v18以上)

ターミナル (Mac: Terminal.app / Windows: PowerShell or WSL)

Anthropicのアカウント (claude.aiで作れる)

手順:

```
npm install -g @anthropic-ai/claude-code
```

これ1行です。これでインストール完了。

次にプロジェクトのフォルダに移動して：

```
cd あなたのプロジェクトフォルダ
claude
```

`claude` と打つだけで起動します。初回はAnthropicアカウントの認証が求められるので、ブラウザが開いたらログインして許可するだけ。

ここまで5分もかかりません。

「ターミナル怖い」という人も安心してください。必要なのは `cd` (フォルダ移動) と `claude` (起動) の2コマンドだけ。これ以上複雑なターミナル操作は、Claude Code自身がやってくれます。

最初の使い方——何を指示すればいいか

インストールできた。 `claude` と打って起動した。さて何を言えばいいか。

ここで「なんか作って」と漠然と指示するのは一番もったいない使い方です。Claude Codeの実力を引き出すには、まず「今あるプロジェクトの中で具体的に何をしてほしいか」を伝えることが大事。

初心者におすすめの最初の一言：

このプロジェクトの構成を説明して

これだけ。Claude Codeはフォルダ内のファイルを全部読んで、「このプロジェクトは〇〇で、〇〇のフレームワークを使っていて、エントリーポイントは〇〇で…」と構造を説明してくれます。

自分が書いたコードでも、他人から引き継いだコードでも、OSSのリポジトリでも、最初にこれを聞くだけでプロジェクト全体の理解が一気に進みます。

次のステップとして：

READMEを読んで、このアプリをローカルで動かすのに必要な手順を教えて

`src/components/Header.tsx` にバグがありそう。調べて修正して

このプロジェクトにダークモードを追加して

こんな感じで、具体的な指示を出すほどClaude Codeは的確に動きます。

CLAUDE.md——これを知らないと損をする

Claude Codeを使う上で、最も重要な機能のひとつが `CLAUDE.md` です。

これはプロジェクトのルートに置くテキストファイルで、「このプロジェクトで Claude Codeがどう振る舞うべきか」を定義するもの。いわば **AIへの永続的な指示書** です。

たとえばこんなことを書きます：

```
# CLAUDE.md
```

```
このプロジェクトはNext.js 14 + TypeScript + Tailwind CSS。  
テストはvitestを使う。  
コミットメッセージはConventional Commitsに従う。  
日本語でコメントを書く。  
新しいファイルを作る時は必ず既存のファイルの命名規則に従う。
```

これを一度書いておくだけで、Claude Codeは毎回この指示を参照してくれます。

何がすごいかというと、「毎回同じ注意点を伝え直す手間」がゼロになること。ChatGPTだと新しい会話のたびに「TypeScriptで書いて」「テストも書いて」「日本語コメントで」と繰り返す必要がある。CLAUDE.mdがあれば、それが全部自動で適用される。

心当たりありませんか？ 毎回同じ前提条件を説明し直している人。CLAUDE.mdひとつで解決します。

Plan Mode——いきなりコードを書かせない

これは初心者が一番やりがちなミスです。

「〇〇を作って」と指示すると、Claude Codeはいきなりコードを書き始めます。でも複雑なタスクほど、最初に「どう作るか」を設計してから書き始めたほうが圧倒的にいい結果になる。

そこで使うのが**Plan Mode**。

Shift + Tab

これでPlan Modeに切り替わります。Plan Modeでは、Claude Codeはコードを書かずに「設計」だけをしてくれる。

たとえば「認証機能を追加して」と指示すると：

どのライブラリを使うか

どのファイルを変更するか

どういう順番で実装するか

考慮すべきエッジケースは何か

これを先に出してくれる。設計を確認して「OK、この方針で進めて」と言えば、そこから実装に入る。

Plan Modeを使うだけで、無駄なやり直しが激減します。トークン（使用量）の節約にもなるし、仕上がりの品質も上がる。「急がば回れ」がClaude Codeでは本当に効きます。

サブエージェントでコンテキストを守る

Claude Codeには「コンテキストウィンドウ」の上限があります。簡単に言うと、一度に覚えていられる情報量に限界がある。

長いセッションを続けていると、最初に伝えた指示や序盤で読んだファイルの内容を忘れ始めます。これがトークンの限界です。

対策として強力なのがサブエージェント。

Claude Codeは内部的に別のAIエージェントを起動して、調査や分析を任せることができます。サブエージェントが使った情報は親のコンテキストを圧迫しない。つまり「調べものは部下に任せて、自分のメモリを空けておく」ことができる。

実際にはClaude Codeが自動的にサブエージェントを使ってくれることが多いですが、明示的に指示することもできます：

```
このプロジェクトのテストカバレッジを調べて。  
サブエージェントを使って各ディレクトリごとに分析して
```

大規模なプロジェクトほど、この機能の恩恵は大きくなります。

/compact——コンテキストが溢れる前に

サブエージェントと合わせて覚えておきたいのが `/compact` コマンド。

```
/compact
```

これを打つと、Claude Codeが今までの会話を要約して、コンテキストを圧縮してくれます。

長時間作業していて「なんかClaude Codeの精度が落ちてきたな」と感じたら、それはコンテキストが溢れかけているサインです。 `/compact` でリフレッシュするだけで、精度が戻ります。

目安としては、30分～1時間ごとに1回。もしくは大きなタスクが一段落したタイミングで打つのがおすすめです。

MCP——Claude Codeの能力を拡張する

MCP（Model Context Protocol）は、Claude Codeに「外部ツールとの接続能力」を追加する仕組みです。

たとえば：

GitHub MCPを入れると、Claude CodeからPRの作成やレビューができる

Slack MCPを入れると、チャンネルの内容を読んでタスクに反映できる

Figma MCPを入れると、デザインを見ながらコードを書ける

ブラウザMCPを入れると、Webページの内容を取得して参照できる

MCPなしのClaude Codeが「優秀な開発者」だとすると、MCPありのClaude Codeは「あらゆるツールを使いこなす開発チーム」になる。

設定もシンプルで、`claude mcp add` コマンドで追加するだけ。

```
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

最初から全部入れる必要はありません。自分の開発フローで「ここ手作業だな」と感じたところから、対応するMCPを1つずつ追加していく。これが一番無理のない進め方です。

トークンの使い方を最適化する

Claude Codeの料金は、使ったトークン量で決まります。ここを意識しないと、あっという間にコストが膨らむ。

トークンを節約するコツ：

1. 指示は1回で明確に出す

「ボタンを追加して」→「色を赤にして」→「位置を右上にして」と3回に分けるより、「右上に赤いボタンを追加して」と1回で言うほうが圧倒的にトークン効率がいい。

2. CLAUDE.mdで繰り返し指示を自動化

前述の通り。毎回伝えている共通指示は全部CLAUDE.mdに書く。

3. Plan Modeで無駄な実装を減らす

いきなり実装して「違う、やり直し」が最もトークンを消費するパターン。

4. /compactを定期的に使う

コンテキストが溢れると、Claude Codeは同じファイルを何度も読み直す。これは純粋な無駄。

5. Sonnetモデルに切り替える

深い思考が不要な単純作業（フォーマット修正、リネームなど）は、`/model` で Sonnetに切り替えると安くて速い。

これらを意識するだけで、同じ予算で2～3倍の作業量をこなせるようになります。

実務での使い方の具体例

理屈は分かった。じゃあ実際にどう使うのか。

ケース1：バグ修正

ユーザーがログインボタンを押しても画面が遷移しない。
原因を調べて修正して

Claude Codeは関連するファイルを自動で特定して、ログを確認して、原因を見つけて、修正して、テストまで書いてくれます。

ケース2：新機能追加

ユーザープロフィールページにアバター画像のアップロード機能を追加して。
S3にアップロードする設計で

Plan Modeで設計を確認 → 承認 → 実装 → テスト。この流れで完結します。

ケース3：リファクタリング

`src/utils/helpers.ts` が500行超えてる。
機能ごとにファイルを分割して、`import`元も全部修正して

手動でやると半日かかる作業が、数分で終わります。

ケース4：コードレビュー

`git diff main...feature-branch` の変更内容をレビューして。
バグ、セキュリティリスク、パフォーマンス問題があれば指摘して

人間のレビュアーと同等以上の精度で、数秒で返ってきます。

初心者がやりがちなミス5選

1. 「なんか作って」と漠然と指示する

→ 具体的であるほどClaude Codeは的確に動く。ゴール、制約、使用技術を明示する。

2. 長時間セッションを/compactなしで続ける

→ 精度がどんどん落ちる。定期的にコンテキストを圧縮する。

3. エラーが出たら自分で直そうとする

→ エラーメッセージをそのままClaude Codeに貼ればいい。AIはエラー解説が得意。

4. CLAUDE.mdを作らない

→ 毎回同じ指示を繰り返す羽目になる。最初に10分かけて書くだけで、以降の全セッションが効率化する。

5. 全部Opusでやろうとする

→ 簡単なタスクはSonnetで十分。使い分けるだけでコストが半分以下になることもある。

初心者から中級者になるための1ヶ月ロードマップ

最後に、Claude Codeの学習ロードマップを整理します。

Week 1：基本操作

インストールして `claude` で起動できるようになる

既存プロジェクトの構造を説明させてみる

簡単な修正（typo修正、スタイル変更など）を指示してみる

Week 2 : CLAUDE.mdとPlan Mode

CLAUDE.mdを書いてプロジェクトに置く

Plan Modeで設計してから実装する流れを身につける

/compactを習慣化する

Week 3 : 実務タスクに投入

バグ修正、機能追加、テスト作成をClaude Codeで実際にやる

エラーが出たらClaude Codeに解決させる

git操作もClaude Codeに任せてみる

Week 4 : MCPと高度な活用

必要なMCPサーバーを1つ追加してみる

サブエージェントを活用した大規模タスクに挑戦

複数セッションの使い分けを試す

1ヶ月もあれば、Claude Codeを「なんとなく使える」から「**開発の主カツール**」に変えられます。

まとめ

Claude Codeは「チャットAI」ではなく「AIエンジニア」。あなたのPC上で直接コードを書く

インストールは `npm install -g @anthropic-ai/claude-code` の1行

CLAUDE.mdで永続的な指示を設定して、毎回の説明を自動化

Plan Modeで設計してから実装する。急がば回れ

/compactでコンテキストを定期リフレッシュ

MCPで外部ツールと連携し、能力を拡張

トークン節約は「1回で明確に指示」「Sonnetとの使い分け」が鉄則

1ヶ月のロードマップで初心者→実務レベルまで到達可能

Claude Codeは「AIでコードを書く」ことのスタンダードを変えたツールです。まだ触れていないなら、今日がはじめるベストなタイミング。